



Estructuras de Datos No Lineales: Árboles

✦ "La información bien organizada no solo se almacena — se comprende, se navega y se transforma."

Agenda de la sesión

¿Qué aprenderemos hoy?

01

Actividad rompehielo

Conectamos con el tema desde nuestra experiencia cotidiana.

02

Estructuras no lineales

Qué son, por qué importan y dónde las usamos en la vida real.

03

Concepto y terminología de árboles

Definición formal, analogías y vocabulario técnico completo.

04

Representación gráfica y práctica

Cómo dibujar árboles, ejemplos reales y actividades interactivas.

05

Evaluación y cierre

Quiz rápido, resumen visual y próximos temas.



 Rompehielo · 2-3 minutos

¡Arranquemos con una pregunta!



Participación en el chat: Responde ahora mismo en el chat de la clase antes de seguir leyendo.



Si tuvieras que organizar todas las carpetas y archivos de tu computadora desde cero... ¿cómo lo harías? ¿Qué estructura usarías?

Piensa en cómo guardas tus tareas, proyectos, imágenes y descargas. ¿Hay un orden jerárquico? ¿Una carpeta "principal" que contiene todas las demás? Esa intuición natural es exactamente el concepto que exploraremos hoy: los **árboles**.

Objetivos de aprendizaje

Al finalizar esta sesión, podrás...



Comprender

Qué son las estructuras de datos no lineales y su rol en programación.



Diferenciar

Estructuras lineales vs. no lineales con ejemplos concretos.



Identificar

Los componentes y la terminología técnica de un árbol.



Construir

Representaciones gráficas básicas y complejas de árboles.

Introducción al tema

¿Por qué necesitamos estructuras no lineales?

Las estructuras lineales (arrays, listas, pilas, colas) son excelentes para datos secuenciales. Pero el mundo real rara vez es lineal: los datos tienen **jerarquías, relaciones y ramificaciones** que las estructuras lineales no pueden representar eficientemente.



Sistemas de archivos

Carpetas dentro de carpetas:
estructura jerárquica natural.



Bases de datos

Índices en árboles B+ para
búsquedas ultrarrápidas.



Motores de búsqueda

Google indexa la web con
estructuras de árbol invertido.



Inteligencia Artificial

Árboles de decisión en modelos de
machine learning.

Comparación fundamental

Lineal vs. No Lineal

Estructuras Lineales

Los elementos se organizan en una **única secuencia**. Cada elemento tiene como máximo un predecesor y un sucesor.

- Array: [5, 12, 8, 3, 9]
- Lista enlazada: $A \rightarrow B \rightarrow C \rightarrow D$
- Pila (Stack): LIFO
- Cola (Queue): FIFO

 Acceso: $O(n)$ en el peor caso

Estructuras No Lineales

Los elementos se conectan en **múltiples dimensiones**. Un nodo puede tener varios sucesores, formando jerarquías o redes.

- Árbol: jerarquía con raíz única
- Grafo: nodos con múltiples conexiones
- Árbol binario de búsqueda
- Árbol B, AVL, Red-Black

 Búsqueda: $O(\log n)$ en árboles balanceados



Concepto central

¿Qué es un Árbol en programación?

Un **árbol** es una estructura de datos no lineal y jerárquica compuesta por **nodos** conectados mediante **aristas** (edges). Existe exactamente un nodo especial llamado **raíz** desde el cual se puede alcanzar cualquier otro nodo siguiendo un único camino.



Definición formal: Un árbol con n nodos tiene exactamente $n-1$ aristas. No existen ciclos.

Analogía natural

- **Raíz del árbol** → nodo raíz
- **Ramas** → aristas/conexiones
- **Hojas** → nodos terminales
- **Tronco principal** → camino desde la raíz

Los componentes de un árbol nodo a nodo



Nodo Raíz

El nodo superior del árbol. **No tiene padre**. Es el punto de entrada a toda la estructura. Ejemplo: C : / en Windows.



Nodo Padre

Nodo que tiene al menos un hijo. Es el **predecesor directo** de otro nodo. Ejemplo: Documentos es padre de Tareas.



Nodo Hijo

Nodo que desciende directamente de otro. Todo nodo (excepto la raíz) tiene **exactamente un padre**.



Nodos Hermanos

Nodos que comparten el **mismo padre**. Están al mismo nivel jerárquico. Ejemplo: Tareas y Proyectos.



Nodo Hoja

Nodo sin hijos. Es un **nodo terminal**. Representa el final de una rama. Ejemplo: un archivo dentro de una carpeta.



Nodo Interno

Nodo que **no es hoja ni raíz**. Tiene al menos un hijo y un padre. Conecta la estructura verticalmente.

Nivel, Profundidad, Altura y Subárbol

Nivel de un nodo

Distancia desde la raíz. La raíz está en el **nivel 0**. Sus hijos en nivel 1, nietos en nivel 2, etc.

Profundidad

Número de aristas desde la **raíz hasta un nodo específico**. La raíz tiene profundidad 0.

Altura del árbol

Número de aristas en el **camino más largo** desde la raíz hasta una hoja. Mide el "tamaño" vertical del árbol.

Subárbol

Cualquier nodo con todos sus descendientes forma un **subárbol**. Todo árbol puede verse como un conjunto de subárboles.

Ejemplo numérico:

En un árbol con 4 niveles (0, 1, 2, 3):

- Altura = 3
- Nodo en nivel 2 → profundidad = 2
- Un nodo hoja en nivel 3 → profundidad = 3 = altura

 **¿Sabías que?** La altura de un árbol perfectamente balanceado con n nodos es $\lfloor \log_2 n \rfloor$. ¡Por eso las búsquedas son tan rápidas!

¿Cómo se dibuja un árbol?

Existen convenciones visuales estándar para representar árboles en informática. Seguir las garantiza que cualquier persona pueda leer e interpretar correctamente la estructura.



1. Dibuja los nodos

Cada nodo se representa con un **círculo** que contiene su etiqueta o valor.



2. Conecta con aristas

Las **líneas** entre nodos representan la relación padre-hijo (aristas). Siempre hacia abajo.



3. Raíz al centro-arriba

La **raíz siempre se coloca en la parte superior**, centrada. Los niveles se distribuyen hacia abajo.



4. Mismos niveles horizontales

Todos los nodos del mismo nivel se colocan en la **misma fila horizontal** para mayor claridad.

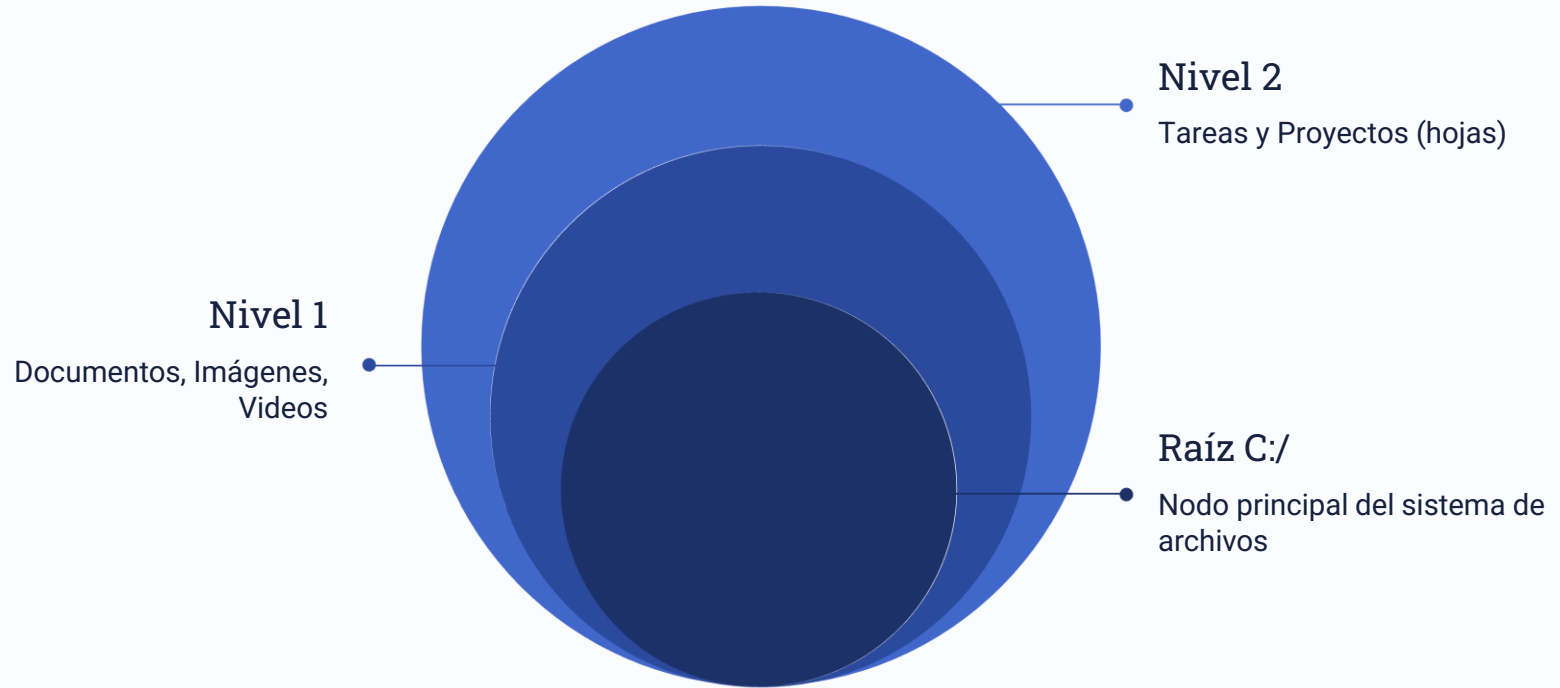
  **Regla clave:** En un árbol NO hay ciclos ni nodos con más de un padre. Si los hay, ya no es un árbol, ¡es un grafo!

Ejemplo práctico guiado

Árbol de directorios de una computadora

Estructura del sistema de archivos

```
C:/ ← Raíz (nivel 0)
|— Documentos (nivel 1)
| |— Tareas (nivel 2)
```



Este árbol tiene **6 nodos** en total, **5 aristas**, una **altura de 2**, y **4 nodos hoja**. Observa cómo la raíz **C:/** está en la cima y los nodos se ramifican hacia abajo siguiendo la jerarquía de carpetas.

 Mini reto colaborativo · 10 minutos

¡Construye tu propio árbol!

Situación real

Una universidad tiene la siguiente estructura organizativa:

- La **Universidad** tiene dos Facultades: Ingeniería y Ciencias.
- La Facultad de **Ingeniería** tiene: Sistemas, Civil y Mecánica.
- La Facultad de **Ciencias** tiene: Matemáticas y Física.
- La carrera de **Sistemas** tiene las asignaturas: Programación I y Programación II.

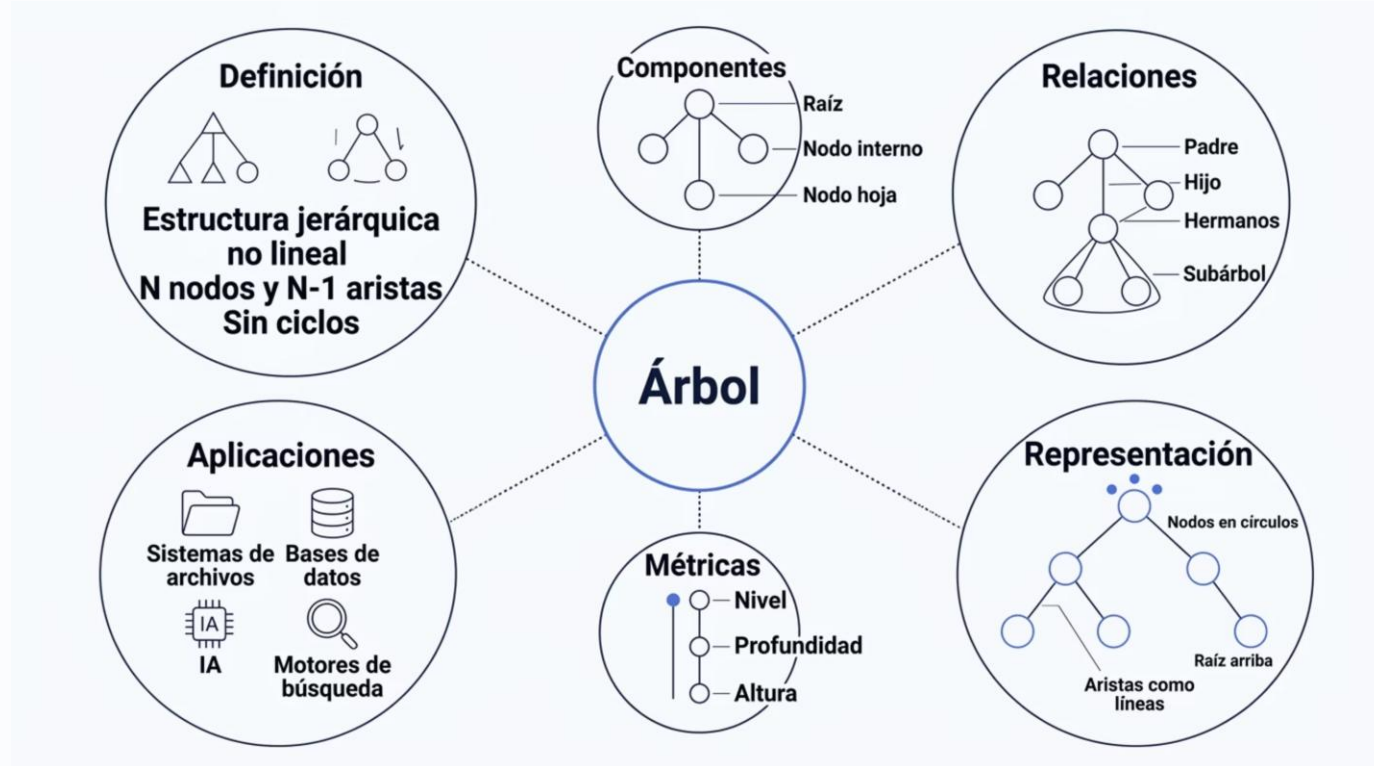
Tu tarea

1. Dibuja el árbol en papel, Jamboard, Miro o en el chat con caracteres ASCII.
2. Identifica: raíz, hojas, nodos internos, altura y nivel de cada nodo.
3. Comparte tu resultado al terminar. ¡Compararemos versiones!



Tienes **5 minutos** para construir el árbol y **5 minutos** para compartirlo.

Todo lo que aprendimos, de un vistazo



Los árboles son la base de estructuras más avanzadas como árboles binarios de búsqueda, árboles AVL, montículos (heaps) y muchas más que explorarás en los próximos temas del curso.

Aplicaciones futuras

¿Dónde usarás los árboles en tu carrera?



🌲 Árbol Binario de Búsqueda (BST)

Próxima sesión: cada nodo tiene máximo 2 hijos. La base de los motores de búsqueda y los índices de bases de datos.



⚖️ Árboles Balanceados (AVL, Red-Black)

Garantizan $O(\log n)$ en búsqueda, inserción y eliminación. Usados en Java TreeMap, C++ STL map y sistemas operativos.



🤖 Árboles de Decisión en IA

Algoritmos de clasificación y regresión en Machine Learning. Cada nodo interno es una condición; cada hoja, una decisión.



🔧 Árboles de Sintaxis (AST)

Los compiladores e intérpretes usan árboles de sintaxis abstracta para analizar y ejecutar tu código fuente.